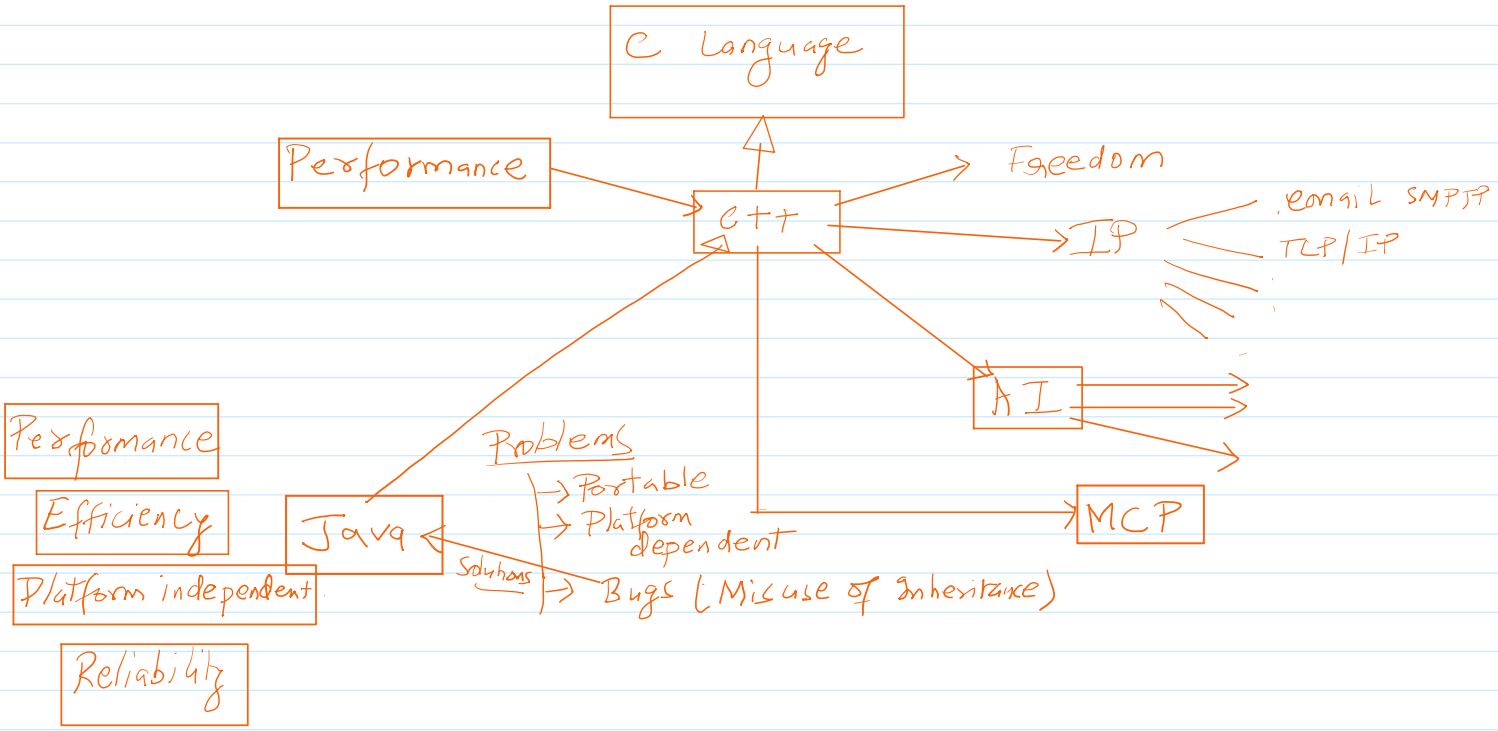
Pointers in C language

26 October 2026 18:31



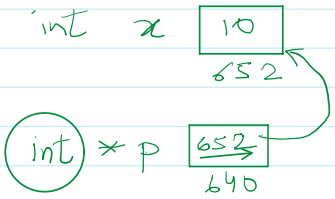
What is a pointer in C?
A **pointer is a variable that stores the memory address of another object or variable,** such as integer, character, array element, or structure.

Normally a variable stores a value. A pointer stores the address where a value is located.

Example:

```c
#include<stdio.h>
int main(int argc, char const *argv[])
{
    /* code */
    int x = 10;
    int *p = &x;
    printf("\nValue of x: %d",x);
    printf("\nValue of p: %u",p);
    printf("\nAddress of x: %u",&x);
    printf("\nAddress of p: %u", &p);
    printf("\nValue at *p: %d", *p);

    return 0;
}
```



& ← address-of-operator (obtains the address of a variable).

* ← Indirection/dereference operator it treats the value as an address of another variable of its own datatype. Therefore int * (integer pointer) can hold the address of int variables only.

[Accesses the object or variable through a

Output:

```
Value of x: 10
Value of p: 4097834652
Address of x: 4097834652
Address of p: 4097834640
```

```
Value of p: 4097834652
Address of x: 4097834652
Address of p: 4097834640
Value at *p: 10
```

[Accesses the object or variable through a pointer].

Declaring and initializing pointers

Syntax:

```
Data-type *pointer_name;

int *ip;
float *fp
char *cp
double *dp;
```

The data type tells Compiler what kind of object the pointer points to.

For example:

```c
#include<stdio.h>
int main(int argc, char const *argv[])
{
    /* code */
    int x = 10;
    int *p = &x;
    int *ip = NULL;
    if(ip != NULL){ // Check before dereferencing
        printf("\n content: %d", *ip);
    }
    else{
        printf("\n ip is NULL");
    }
    printf("\nValue of x: %d",x);
    printf("\nValue of p: %u",p);
    printf("\nAddress of x: %u",&x);
    printf("\nAddress of p: %u", &p);
    printf("\nValue at *p: %d", *p);
    return 0;
}
```

Output:

```
 ip is NULL
Value of x: 10
Value of p: 2348809236
Address of x: 2348809236
Address of p: 2348809224
Value at *p: 10
```

A **NULL** pointer does not point to a valid object and must not be dereferenced.

Modifying a variable using a pointer

One of the most useful feature of pointer is that you can modify the original variable without directly referring to its name.

```c
#include<stdio.h>
int main(int argc, char const *argv[])
{
    /* code */
    int x = 10;
    int *p = &x;
    int *ip = NULL;
    if(ip != NULL){ // Check before dereferencing
        printf("\n content: %d", *ip);
    }
    else{
        printf("\n ip is NULL");
    }
    printf("\nValue of x: %d",x);
    printf("\nValue of p: %u",p);
    printf("\nAddress of x: %u",&x);
    printf("\nAddress of p: %u", &p);
    printf("\nValue at *p: %d", *p);
    *p = 100;  // This is called indirect modification through a pointer.
    printf("\nAfter modification using *p value of x: %d",x);
    return 0;
}
```

Output:

```
    ip is NULL
   Value of x: 10
   Value of p: 2092955652
   Address of x: 2092955652
   Address of p: 2092955640
   Value at *p: 10
   After modification using *p value of x: 100
```

Pointers and functions: call by value vs pointer-based modification

In C function arguments are always passed by value. However, passing a pointer by value lets a function modify the object to which the pointer refers.

```c
#include<stdio.h>
void change(int x){
    x = 100;
}
int main(int argc, char const *argv[])
{
    int n = 10;
    change(n);
    printf("\nn = %d", n);
    return 0;
```

```
}
```

Output:

```
   n = 10
```

The function changes only its local copy of **n**.

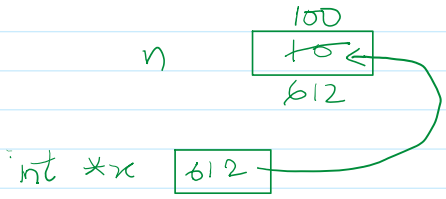
Using a pointer

```
#include<stdio.h>
void change(int *x){
    *x = 100;
}
int main(int argc, char const *argv[])
{
    int n = 10;
    change(&n);
    printf("\nn = %d", n);
    return 0;
}
```

The parameter *x receives a copy of that address

modifies the original variable

passes the address of 'n'



Output:

```
n = 100
```

This is commonly describing as passing an address or using pointer - based modification.
It is not true Call-by-reference in the language formal parameter-passing model.

Swapping two numbers using pointers

This is a classic pointer-programming example.

```
#include<stdio.h>
void swap(int *a, int *b){
    int t = *a;
    *a = *b;
    *b = t;
}
int main(int argc, char const *argv[])
{
    int x = 10;
    int y = 20;
    printf("\nBefore swapping: \n");
    printf("x = %d, y = %d\n", x, y);
    swap(&x, &y);
    printf("\nAfter swapping: \n");
    printf("x = %d, y = %d\n", x, y);

    return 0;
}
```

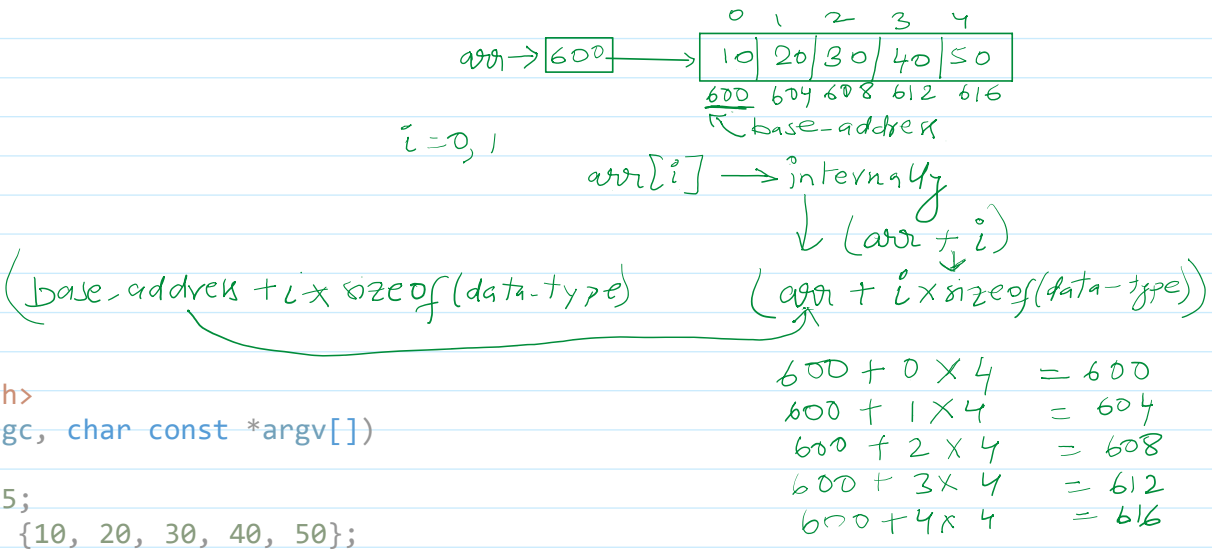
```
Output:
   Before swapping:
   x = 10, y = 20

   After swapping:
   x = 20, y = 10
```

Because the pointer refers to **x** and **y**, the function changes the original variables.

Pointer arithmetic
Pointer arithmetic allows you to move between elements of an array. It is not ordinary arithmetic on raw byte addresses.

The name of array holds the reference or address of first location or 0(zeroth) index.




```c
#include<stdio.h>
int main(int argc, char const *argv[])
{
    int size = 5;
    int arr[] = {10, 20, 30, 40, 50};
    int *p = arr;
    int i;
    printf("\nAddress of each element of arr[]: \n");
    for(i = 0; i < size; i++){
        printf("arr[%d]: %d\n", i, (arr+i));
    }
    return 0;
}
```

```
Output:
   Address of each element of arr[]:
   arr[0]: 880802400
   arr[1]: 880802404
   arr[2]: 880802408
   arr[3]: 880802412
   arr[4]: 880802416
```

```c
#include<stdio.h>
int main(int argc, char const *argv[])
{
    int size = 5;
```

```
    int arr[] = {10, 20, 30, 40, 50};
    int *p = arr;
    int i;

    printf("\nContent of each element of arr[]: \n");
    for(i = 0; i < size; i++){
        printf("arr[%d]: %d\n", i, *(arr+i));
    }

    return 0;
}
```

Output:

```
Content of each element of arr[]:
arr[0]: 10
arr[1]: 20
arr[2]: 30
arr[3]: 40
arr[4]: 50
```

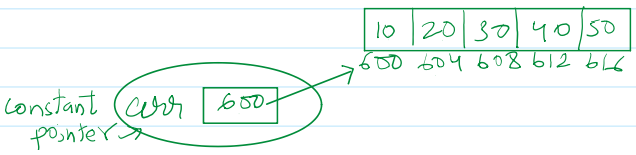
If an int occupies four bytes on a particular system. Advancing an **int*** by one element usually advances its address by four bytes. The size of int is implementation-dependent.

```
#include<stdio.h>
int main(int argc, char const *argv[])
{
    int size = 5;
    int arr[] = {10, 20, 30, 40, 50};
    int *p = arr;
    int i;
    // printf("\nAddress of each element of arr[]: \n");
    // printf("\nContent of each element of arr[]: \n");
    // for(i = 0; i < size; i++){
    //     printf("arr[%d]: %d\n", i, *(arr+i));
    // }
    printf("\nFirst element: %d", *arr);
    printf("\nSecond element: %d", *(arr + 1));
    printf("\nThird element: %d", *(arr + 2));
    return 0;
}
```

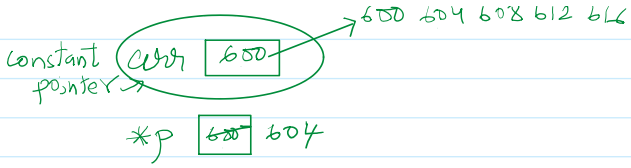
Output:

```
First element: 10
Second element: 20
Third element: 30
```

```
#include<stdio.h>
int main(int argc, char const *argv[])
{
    int size = 5;
    int arr[] = {10, 20, 30, 40, 50};
```



```c
{
    int size = 5;
    int arr[] = {10, 20, 30, 40, 50};
    int *p = arr;
    int i;
    printf("\nFirst element: %d", *p);
    printf("\nSecond element: %d", *(p + 1));
    printf("\nThird element: %d", *(p + 2));
    // arr++; constant pointer
    p++; //pointing to the second element
    printf("\nAfter p++ = %d\n", *p);
    return 0;
}
```



Output:

```
First element: 10
Second element: 20
Third element: 30
After p++ = 20
```

```c
#include<stdio.h>
int main(int argc, char const *argv[])
{
    int size = 5;
    int arr[] = {10, 20, 30, 40, 50};
    int i, *p = arr;
    for(i = 0; i < size - 1; p++, i++); //pointer increment
    for(;i >= 0; i--, p--){ //pointer decrement
        printf("\n%d", *p);
    }
    return 0;
}
```

Output:

```
50
40
30
20
10
```

Pointer arithmetic must stay within the same array object or one position past its end. The **one-past-the-end** pointer can be used for comparisons and iteration boundaries, but it must not be dereferenced.

```c
#include<stdio.h>
void display(int *p, int size){
    int i;
    for(i = 0; i < size - 1; p++, i++); //pointer increment
    for(;i >= 0; i--, p--){ //pointer decrement
        printf("\n%d", *p);
    }
```

```
}
int main(int argc, char const *argv[])
{
    int size = 5;
    int arr[] = {10, 20, 30, 40, 50};
    display(arr, size);

    return 0;
}
```

```
Output:
   50
   40
   30
   20
   10
```